

Natalie Leung
014790932
5 October 2026

Data 260 Homework 5

Personal Configuration and Domain

SID4	0932
PORT_BASE	8032
PREFIX	s0932
SEED	0932
VERIFY_SEED	260932
DOMAIN_ID	4 (open-source package vulnerabilities)

Hardware: nvidia geforce 4070, msi cpu

Local model used: ollama qwen3:8b

Tagged commit hash repo with most updated codes (submission on canvas only):

<https://github.com/n-k-leung/data260-0932>

Part 1

I. Database

- Create a second table for a related entity type (e.g., for a book/author-style setup, this second entity plays the role of the "author"). It should include id (primary key), a primary text field, a secondary text field, a unique field (e.g., email or code), and timestamps.

```
class Vendor (Base):
    __tablename__ = "vendors"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String(255), nullable=False)
    industry = Column(String(100), nullable=False)
    contact_email = Column(String(255), nullable=False, unique=True)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), server_default=func.now(), onupdate=func.now())

    #all vulnerabilities from specific vendor
    vulnerabilities = relationship("Vul", back_populates="vendor")
```

- Your primary domain entity's table should contain id (primary key), primary field, a unique field (e.g., an ID/code), a numeric field with a sensible default (e.g., an available-count style field), a foreign key to the related entity, and timestamps. Use appropriate data types and constraints.

```

class Vul(Base):
    __tablename__ = "vulnerabilities"
    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    package_name = Column(String(255), nullable=False)
    severity = Column(String(255), nullable=False)
    advisories = relationship(
        "Advisory",
        primaryjoin="Vul. id == Advisory.vul_id",
        foreign_keys="Advisory.vul_id",
        viewonly=True,
    )
    vul_code = Column(String(50), nullable=False, unique=True)
    report_count = Column(Integer, nullable=False, default=0)
    vendor_id = Column(Integer, ForeignKey("vendors.id"), nullable=False)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), server_default=func.now(), onupdate=func.now())

    #which vendor this vul is for
    vendor = relationship("Vendor", back_populates="vulnerabilities")

```

- Do not store plain-text passwords. Prevent deletion of a related-entity record that still has associated primary-entity records, unless you deliberately implement and document cascade behavior.

```

def create_user(db: Session, payload: schema.UserCreate):
    hashed = hash_password(payload.password)
    user = models.User(name=payload.name, email=payload.email, password_hash=hashed)
    db.add(user)
    db.commit()
    db.refresh(user)
    return user

```

```

def delete_vendor(db: Session, vendor_id: int):
    vendor = get_vendor(db, vendor_id)
    if not vendor:
        return None
    # do not delete a vendor that still has vulnerabilities pointing to it
    has_children = db.query(models.Vul).filter(models.Vul.vendor_id == vendor_id).first()
    if has_children:
        raise ValueError("Vendor still has vulnerabilities, cannot delete")
    db.delete(vendor)
    db.commit()
    return vendor

```

II. API Implementation

- Implement CRUD endpoints for both entities.
- For the related entity: add new records, retrieve all with pagination, get individual records, update, delete.

```

def create_vendor(db: Session, payload: schema.VendorCreate):
    vendor = models.Vendor(name=payload.name, industry=payload.industry, contact_email=payload.contact_email)
    db.add(vendor)
    db.commit()
    db.refresh(vendor)
    return vendor

def get_vendors(db: Session, skip: int = 0, limit: int = 50):
    return db.query(models.Vendor).order_by(models.Vendor.id.asc()).offset(skip).limit(limit).all()

def get_vendor(db: Session, vendor_id: int):
    return db.query(models.Vendor).filter(models.Vendor.id == vendor_id).first()

def update_vendor(db: Session, vendor_id: int, payload: schema.VendorUpdate):
    vendor = get_vendor(db, vendor_id)
    if not vendor:
        return None
    vendor.name = payload.name
    vendor.industry = payload.industry
    vendor.contact_email = payload.contact_email
    db.commit()
    db.refresh(vendor)
    return vendor

def delete_vendor(db: Session, vendor_id: int):
    vendor = get_vendor(db, vendor_id)
    if not vendor:
        return None
    # do not delete a vendor that still has vulnerabilities pointing to it
    has_children = db.query(models.Vul).filter(models.Vul.vendor_id == vendor_id).first()
    if has_children:
        raise ValueError("Vendor still has vulnerabilities, cannot delete")
    db.delete(vendor)
    db.commit()
    return vendor

def get_vuls_by_vendor(db: Session, vendor_id: int):
    return db.query(models.Vul).filter(models.Vul.vendor_id == vendor_id).all()

```

- For your primary domain entity: same set - create, list, retrieve, update, delete.

```
#functions for vulnerabilities, follows same logic as useres demo
def create_vul(db: Session, payload: schema.VulCreate):
    vul = models.Vul(package_name=payload.package_name, severity=payload.severity, vul_code = payload.vul_code, vendor_id = payload.vendor_id)
    db.add(vul)
    db.commit()
    db.refresh(vul)
    return vul

def get_vuls(db: Session):
    return db.query(models.Vul).order_by(models.Vul.id.asc()).all()

def get_vul(db: Session, vul_id: int):
    return db.query(models.Vul).filter(models.Vul.id == vul_id).first()

def update_vul(db: Session, vul_id: int, payload: schema.VulUpdate):
    vul = get_vul(db, vul_id)
    if not vul:
        return None
    vul.package_name = payload.package_name
    vul.severity = payload.severity
    vul.vul_code = payload.vul_code
    vul.vendor_id = payload.vendor_id
    db.commit()
    db.refresh(vul)
    return vul

def delete_vul(db: Session, vul_id: int):
    vul = get_vul(db, vul_id)
    if not vul:
        return None
    db.delete(vul)
    db.commit()
    return vul
```

- Include one additional endpoint to get all primary-entity records associated with a specific related-entity record (the relationship query).

```
#relationship query so that all vulnerabilities belong to one vendor
@app.get("/vendors/{vendor_id}/vulnerabilities", response_model=list[schema.VulOut])
def list_vulnerabilities_for_vendor(vendor_id: int, db: Session = Depends(get_db), _session = Depends(require_session)):
    if not crud.get_vendor(db, vendor_id):
        raise HTTPException(status_code=404, detail="Vendor not found")
    return crud.get_vuls_by_vendor(db, vendor_id)
```

- Use Pydantic schemas for request/response validation. Implement proper HTTP status codes and error messages for resource-not-found, validation errors, and constraint violations. Ensure the unique field's format is validated.

```
@app.post("/vuls", response_model=schema.VulOut)
def add_vul(payload: schema.VulCreate, db: Session = Depends(get_db), _session = Depends(require_session)):
    if not crud.get_vendor(db, payload.vendor_id):
        raise HTTPException(status_code=404, detail="Vendor not found")
    try:
        return crud.create_vul(db, payload)
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail="vulnerability code already exists")
```

```

@app.get("/vuls/{vul_id}", response_model=schema.VulOut)
def get_vul(vul_id: int, db: Session = Depends(get_db), _session = Depends(require_session)):
    vul = crud.get_vul(db, vul_id)
    if not vul:
        raise HTTPException(status_code=404, detail="Vulnerability not found")
    return vul

@app.put("/vuls/{vul_id}", response_model=schema.VulOut)
def edit_vul(vul_id: int, payload: schema.VulUpdate, db: Session = Depends(get_db), _session = Depends(require_session)):
    if not crud.get_vendor(db, payload.vendor_id):
        raise HTTPException(status_code=404, detail="Vendor not found")
    try:
        vul = crud.update_vul(db, vul_id, payload)
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail="vulnerability code already exists")
    if not vul:
        raise HTTPException(status_code=404, detail="Vulnerability not found")
    return vul

@app.delete("/vuls/{vul_id}", response_model=schema.VulOut)
def remove_vul(vul_id: int, db: Session = Depends(get_db), _session = Depends(require_session)):
    vul = crud.delete_vul(db, vul_id)
    if not vul:
        raise HTTPException(status_code=404, detail="Vulnerability not found")
    return vul

```

```

#secondary entity for vendors
@app.post("/vendors", response_model=schema.VendorOut)
def add_vendor(payload: schema.VendorCreate, db: Session = Depends(get_db), _session = Depends(require_session)):
    try:
        return crud.create_vendor(db, payload)
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail="Vendor contact_email already exists")

@app.get("/vendors", response_model=list[schema.VendorOut])
def list_vendors(skip: int = 0, limit: int = 50, db: Session = Depends(get_db), _session = Depends(require_session)):
    return crud.get_vendors(db, skip, limit)

@app.get("/vendors/{vendor_id}", response_model=schema.VendorOut)
def get_vendor(vendor_id: int, db: Session = Depends(get_db), _session = Depends(require_session)):
    vendor = crud.get_vendor(db, vendor_id)
    if not vendor:
        raise HTTPException(status_code=404, detail="Vendor not found")
    return vendor

@app.put("/vendors/{vendor_id}", response_model=schema.VendorOut)
def edit_vendor(vendor_id: int, payload: schema.VendorUpdate, db: Session = Depends(get_db), _session = Depends(require_session)):
    try:
        vendor = crud.update_vendor(db, vendor_id, payload)
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail="Vendor contact_email already exists")
    if not vendor:
        raise HTTPException(status_code=404, detail="Vendor not found")
    return vendor

@app.delete("/vendors/{vendor_id}", response_model=schema.VendorOut)
def remove_vendor(vendor_id: int, db: Session = Depends(get_db), session = Depends(require_session)):
    try:
        vendor = crud.delete_vendor(db, vendor_id)
    except ValueError as e:
        raise HTTPException(status_code=409, detail=str(e))
    if not vendor:
        raise HTTPException(status_code=404, detail="Vendor not found")
    return vendor

```

- Submit Postman screenshots of each action tested using the API endpoints. Submit a screenshot of your DB.

Create vulnerability

Home Workspaces Explore Search Postman Sign In Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

- Today
 - POST http://localhost:8032/vuls
 - POST http://localhost:8032/auth/login
 - POST http://localhost:8032/vuls
 - POST http://localhost:8032/auth/register
 - POST http://localhost:8032/auth/register
- October 3
 - POST http://localhost:8032/auth/register
- October 2
 - POST http://localhost:8032/auth/register
 - POST http://localhost:8032/auth/register
 - POST http://localhost:8032/auth/register
 - POST http://localhost:8032/auth/register
 - GET http://localhost:8032/vulnerabilities
 - POST http://localhost:8032/vulnerabilities
 - GET http://localhost:8032/vulnerabilities
 - GET http://localhost:8032/vendors

Create collections in Postman
Use collections to save your requests and share them with others.

http://localhost:8032/vuls

POST http://localhost:8032/vuls

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "package_name": "requests",
3   "severity": "high",
4   "vul_code": "sk394",
5   "vendor_id": 1
6 }
7
```

Body Cookies (1) Headers (5) Test Results 200 OK 26 ms 312 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 5,
3   "package_name": "requests",
4   "severity": "high",
5   "vul_code": "sk394",
6   "report_count": 0,
7   "vendor_id": 1,
8   "created_at": "2026-10-05T10:43:19",
9   "updated_at": "2026-10-05T10:43:19"
10 }
```

Get all vulnerabilities

HomeWorkspacesExplore

Search Postman

Sign InCreate Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History

NewImport

POST http://GET http://POST http://[CONFLICT] POST http://[CONFLICT] GET http://POST http://POST http://POST http://GET http://

http://localhost:8032/vuls

Today

GET http://localhost:8032/vuls

GET http://localhost:8032/vuls/1

GET http://localhost:8032/vuls/1

GET http://localhost:8032/vuls

GET http://localhost:8032/vuls

POST http://localhost:8032/vuls

POST http://localhost:8032/vuls

POST http://localhost:8032/vuls

POST http://localhost:8032/vuls

POST http://localhost:8032/vuls

POST http://localhost:8032/auth/login

POST http://localhost:8032/vuls

POST http://localhost:8032/auth/register

POST http://localhost:8032/auth/register

October 3

POST http://localhost:8032/auth/register

October 2

POST http://localhost:8032/auth/register

POST http://localhost:8032/auth/register

POST http://localhost:8032/auth/register

POST http://localhost:8032/auth/register

GET http://localhost:8032/vulnerabilities

POST http://localhost:8032/vulnerabilities

GET http://localhost:8032/vulnerabilities

GET http://localhost:8032/vendors

POST http://localhost:8032/vendors

GET http://localhost:8032/vendors

GET http://localhost:8032/vendors

POST http://localhost:8032/auth/login

GET http://localhost:8032/auth/login

GET http://localhost:8032/auth/login

GET http://localhost:8032/auth/login

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

http://localhost:8032/vuls

GET

Send

Params

Authorization

Headers (7)

Body

Pre-request Script

Tests

Settings

Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body

Cookies (1)

Headers (5)

Test Results

200 OK14 ms830 B

Save Response

PrettyRawPreviewVisualizeJSON

```
1
2
3  {
4    "id": 1,
5    "package_name": "nodejs",
6    "severity": "high",
7    "vul_code": "23s67es",
8    "report_count": 0,
9    "vendor_id": 1,
10   "created_at": "2026-10-03T19:45:26",
11   "updated_at": "2026-10-03T19:45:26"
12 },
13 {
14   "id": 2,
15   "package_name": "python",
16   "severity": "medium",
17   "vul_code": "s9f6es",
18   "report_count": 0,
19   "vendor_id": 2,
20   "created_at": "2026-10-03T19:46:37",
21   "updated_at": "2026-10-03T19:46:37"
22 },
23 {
24   "id": 3,
25   "package_name": "test",
26   "severity": "low",
27   "vul_code": "asdf95",
28   "report_count": 0,
29   "vendor_id": 1,
30   "created_at": "2026-10-05T10:18:58",
31   "updated_at": "2026-10-05T10:18:58"
32 }
```

Get one vulnerability

Home Workspaces Explore Search Postman

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

Today

- GET http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls
- GET http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/auth/login
- POST http://localhost:8032/vuls
- POST http://localhost:8032/auth/register
- POST http://localhost:8032/auth/register

October 3

- POST http://localhost:8032/auth/register

October 2

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

POST http:// GET http:// POST http:// [CONFLICT] POST http:// [CONFLICT] GET http:// POST http:// POST http:// POST http:// GET http://

http://localhost:8032/vuls

GET http://localhost:8032/vuls/1

Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies (1) Headers (5) Test Results

200 OK 12 ms 312 B Save Response

Pretty Raw Preview Visualize JSON

```

1 {
2   "id": 1,
3   "package_name": "nodejs",
4   "severity": "high",
5   "vul_code": "23s67es",
6   "report_count": 0,
7   "vendor_id": 1,
8   "created_at": "2026-10-03T19:45:26",
9   "updated_at": "2026-10-03T19:45:26"
10 }

```

Update vulnerability

Home Workspaces Explore Search Postman

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

Today

- PUT http://localhost:8032/vuls/3
- PUT http://localhost:8032/vuls/3
- PUT http://localhost:8032/vuls/3
- PUT http://localhost:8032/vuls/3
- GET http://localhost:8032/vendors/1/vulnerabilities
- GET http://localhost:8032/vendors/1/vuls
- DEL http://localhost:8032/vendors/3
- PUT http://localhost:8032/vendors/3
- GET http://localhost:8032/vendors/1
- GET http://localhost:8032/vendors
- POST http://localhost:8032/vendors
- DEL http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls
- GET http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls

http://localhost:8032/vuls/5

PUT http://localhost:8032/vuls/3

Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON

Beautify

Body Cookies (1) Headers (5) Test Results

200 OK 17 ms 315 B Save Response

Pretty Raw Preview Visualize JSON

```

1 {
2   "package_name": "updated",
3   "severity": "critical",
4   "vul_code": "srf3r",
5   "vendor_id": 1
6 }

```

Body

```

1 {
2   "id": 3,
3   "package_name": "updated",
4   "severity": "critical",
5   "vul_code": "srf3r",
6   "report_count": 0,
7   "vendor_id": 1,
8   "created_at": "2026-10-05T10:18:58",
9   "updated_at": "2026-10-05T10:54:58"
10 }

```

Delete vulnerability

Home Workspaces Explore Search Postman Sign In Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

Today

- DEL http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls
- GET http://localhost:8032/vuls/1
- GET GET http://localhost:8032/vuls/1
- GET GET http://localhost:8032/vuls
- GET GET http://localhost:8032/vuls
- POST GET http://localhost:8032/vuls
- POST GET http://localhost:8032/vuls
- POST GET http://localhost:8032/vuls
- POST GET http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/auth/login
- POST http://localhost:8032/vuls
- POST http://localhost:8032/auth/register
- POST http://localhost:8032/auth/register

October 3

- POST http://localhost:8032/auth/register

October 2

- POST http://localhost:8032/auth/register
- POST http://localhost:8032/auth/register
- POST http://localhost:8032/auth/register
- POST http://localhost:8032/auth/register

http://localhost:8032/vuls/5

DELETE http://localhost:8032/vuls/5 Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Body Cookies (1) Headers (5) Test Results 200 OK 16 ms 312 B Save Response

Pretty Raw Preview Visualize JSON Beautify

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Create vendor

Home Workspaces Explore Search Postman

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

history New Import

Today

- POST http://localhost:8032/vendors
- DEL http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls
- GET http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls
- GET http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/auth/login
- POST http://localhost:8032/vuls
- POST http://localhost:8032/auth/register
- POST http://localhost:8032/auth/register

October 3

- POST http://localhost:8032/auth/register

October 2

- POST http://localhost:8032/auth/register
- POST http://localhost:8032/auth/register

http://localhost:8032/vuls/5

POST http://localhost:8032/vendors

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

Body

```
1 {
2   "name": "test2",
3   "industry": "hardware",
4   "contact_email": "vendor@test.com"
5 }
6
```

Body Cookies (1) Headers (5) Test Results

200 OK 17 ms 289 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 3,
3   "name": "test2",
4   "industry": "hardware",
5   "contact_email": "vendor@test.com",
6   "created_at": "2026-10-05T10:48:19",
7   "updated_at": "2026-10-05T10:48:19"
8 }
```

Get all vendors

Home Workspaces Explore Search Postman

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

Today

- GET http://localhost:8032/vendors
- POST http://localhost:8032/vendors
- DEL http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls
- GET http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/auth/login
- POST http://localhost:8032/vuls
- POST http://localhost:8032/auth/register
- POST http://localhost:8032/auth/register

October 3

- POST http://localhost:8032/auth/register

October 2

- POST http://localhost:8032/auth/register
- POST http://localhost:8032/auth/register
- POST http://localhost:8032/auth/register
- POST http://localhost:8032/auth/register
- GET http://localhost:8032/vulnerabilities
- POST http://localhost:8032/vulnerabilities
- GET http://localhost:8032/vulnerabilities
- GET http://localhost:8032/vendors
- GET http://localhost:8032/vendors

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

http://localhost:8032/vuls/5

GET http://localhost:8032/vendors

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

Body

```
1 {
2   "name": "test2",
3   "industry": "hardware",
4   "contact_email": "vendor@test.com"
5 }
6
```

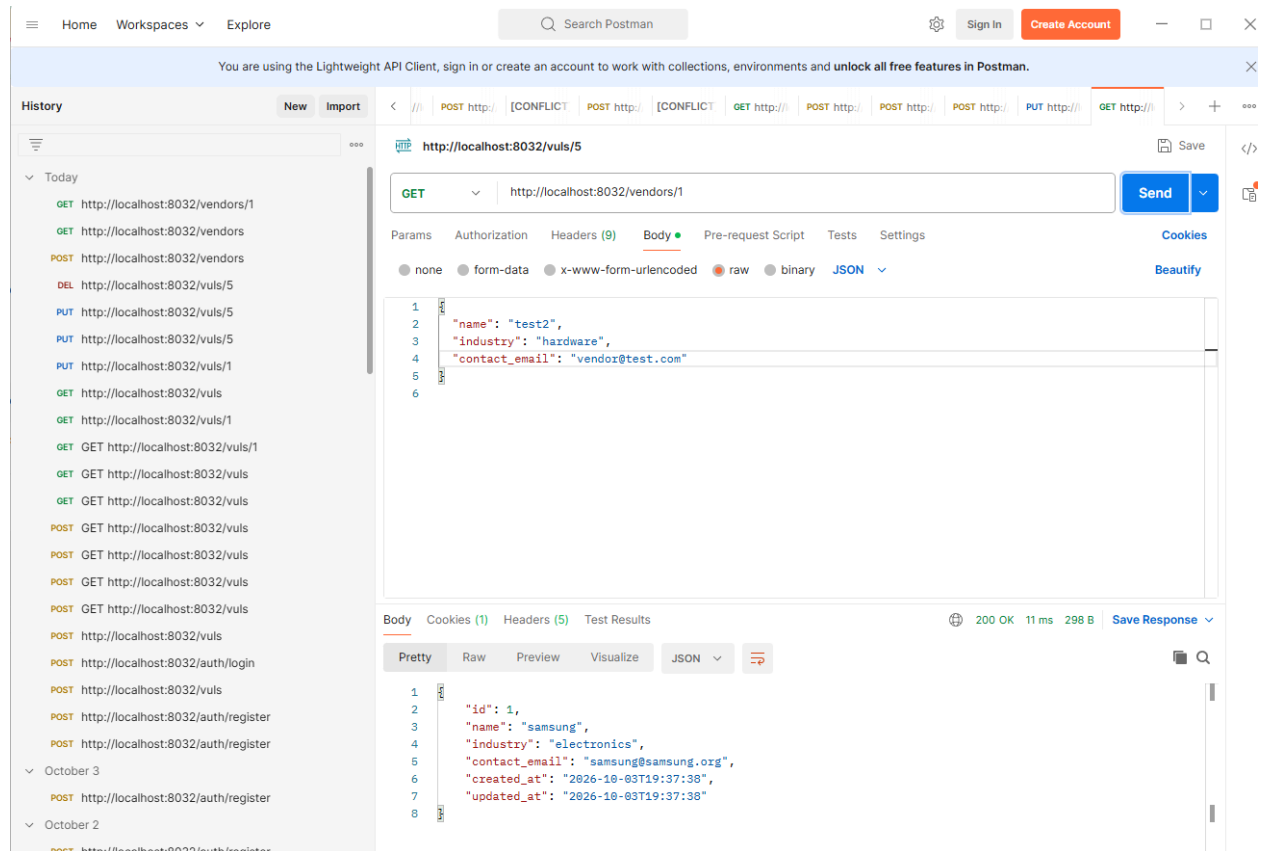
Body Cookies (1) Headers (5) Test Results

200 OK 14 ms 604 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 1,
3   "name": "samsung",
4   "industry": "electronics",
5   "contact_email": "samsung@samsung.org",
6   "created_at": "2026-10-03T19:37:38",
7   "updated_at": "2026-10-03T19:37:38"
8 },
9 {
10  "id": 2,
11  "name": "apple",
12  "industry": "electronics",
13  "contact_email": "appleco@apple.org",
14  "created_at": "2026-10-03T19:46:21",
15  "updated_at": "2026-10-03T19:46:21"
16 },
17 {
18  "id": 3,
19  "name": "test2",
20  "industry": "hardware",
21  "contact_email": "vendor@test.com",
22  "created_at": "2026-10-05T10:48:19",
23  "updated_at": "2026-10-05T10:48:19"
24 }
25
26
```

Get one vendor



Update vendor

Home Workspaces Explore Search Postman Sign In Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

Today

- PUT http://localhost:8032/vendors/3
- GET http://localhost:8032/vendors/1
- GET http://localhost:8032/vendors
- POST http://localhost:8032/vendors
- DEL http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls
- GET http://localhost:8032/vuls/1
- GET GET http://localhost:8032/vuls/1
- GET GET http://localhost:8032/vuls
- GET GET http://localhost:8032/vuls
- POST GET http://localhost:8032/vuls
- POST GET http://localhost:8032/vuls
- POST GET http://localhost:8032/vuls
- POST GET http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/auth/login
- POST http://localhost:8032/vuls
- POST http://localhost:8032/auth/register
- POST http://localhost:8032/auth/register
- October 3
- POST http://localhost:8032/auth/register

http://localhost:8032/vuls/5

PUT http://localhost:8032/vendors/3

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "test",
3   "industry": "cybersecurity",
4   "contact_email": "vendor@test.com"
5 }
6
```

Body Cookies (1) Headers (5) Test Results 200 OK 18 ms 293 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 3,
3   "name": "test",
4   "industry": "cybersecurity",
5   "contact_email": "vendor@test.com",
6   "created_at": "2026-10-05T10:48:19",
7   "updated_at": "2026-10-05T10:50:34"
8 }
```

Delete vendor

Home Workspaces Explore Search Postman Sign In Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

Today

- DEL http://localhost:8032/vendors/3
- PUT http://localhost:8032/vendors/3
- GET http://localhost:8032/vendors/1
- GET http://localhost:8032/vendors
- POST http://localhost:8032/vendors
- DEL http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/5
- PUT http://localhost:8032/vuls/1
- GET http://localhost:8032/vuls
- GET http://localhost:8032/vuls/1
- GET GET http://localhost:8032/vuls/1
- GET GET http://localhost:8032/vuls
- GET GET http://localhost:8032/vuls
- POST GET http://localhost:8032/vuls
- POST GET http://localhost:8032/vuls
- POST GET http://localhost:8032/vuls
- POST http://localhost:8032/vuls
- POST http://localhost:8032/auth/login
- POST http://localhost:8032/vuls
- POST http://localhost:8032/auth/register
- POST http://localhost:8032/auth/register
- October 2

http://localhost:8032/vuls/5

DELETE http://localhost:8032/vendors/3

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "test",
3   "industry": "cybersecurity",
4   "contact_email": "vendor@test.com"
5 }
6
```

Body Cookies (1) Headers (5) Test Results 200 OK 20 ms 293 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 3,
3   "name": "test",
4   "industry": "cybersecurity",
5   "contact_email": "vendor@test.com",
6   "created_at": "2026-10-05T10:48:19",
7   "updated_at": "2026-10-05T10:50:34"
8 }
```

Relationship

The screenshot displays the Postman API client interface. The top navigation bar includes 'Home', 'Workspaces', and 'Explore' tabs, along with a search bar and 'Sign In' and 'Create Account' buttons. A notification banner at the top states: 'You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.'

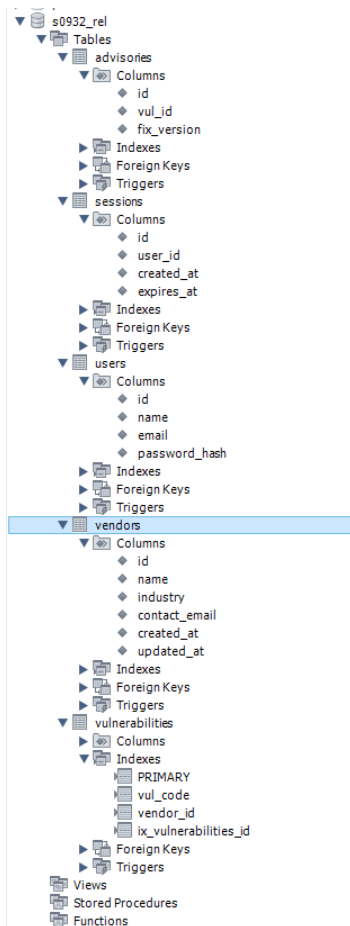
The left sidebar shows a 'History' panel with a list of recent requests. The main workspace is titled 'http://localhost:8032/vuls/5'. The request is a GET method to 'http://localhost:8032/vendors/1/vulnerabilities'. The request body is a JSON object:

```
{
  "name": "test",
  "industry": "cybersecurity",
  "contact_email": "vendor@test.com"
}
```

The response is displayed in the bottom panel, showing a 200 OK status with a response time of 12 ms and a body size of 483 B. The response body is a JSON array of two objects:

```
[
  {
    "id": 1,
    "package_name": "nodejs",
    "severity": "high",
    "vul_code": "23s67es",
    "report_count": 0,
    "vendor_id": 1,
    "created_at": "2026-10-03T19:45:26",
    "updated_at": "2026-10-03T19:45:26"
  },
  {
    "id": 3,
    "package_name": "test",
    "severity": "low",
    "vul_code": "asdf95",
    "report_count": 0,
    "vendor_id": 1,
    "created_at": "2026-10-05T18:18:58",
    "updated_at": "2026-10-05T18:18:58"
  }
]
```

Db screenshot



III. Redux Client

Move your React data layer to Redux Toolkit, integrating with the FastAPI + MySQL backend above.

1. Redux Setup

- Create a Redux store using Redux Toolkit.

```
1 import { configureStore } from "@reduxjs/toolkit";
2 import vulsReducer from "../vulSlice.js";
3
4 export const store = configureStore({
5   reducer: {
6     vuls: vulsReducer,
7   },
8 });
```

- Define a slice to manage the state of your primary domain entity.

```
const vulSlice = createSlice({
  name: "vulnerabilities",
  initialState: { items: [], loading: false, error: null },
  reducers: {},
  extraReducers: (builder) => {
    builder
```

- Include reducers/actions for fetching, adding, updating, and deleting records.

```
// fetch: show loading, then fill items with the list from the server
.addCase(fetchVuls.pending, (state) => {
  state.loading = true;
  state.error = null;
})
.addCase(fetchVuls.fulfilled, (state, action) => {
  state.loading = false;
  state.items = action.payload;
})
.addCase(fetchVuls.rejected, (state, action) => {
  state.loading = false;
  state.error = action.error.message;
})
// create: add the new record returned by the server to the list
.addCase(createVul.fulfilled, (state, action) => {
  state.items.push(action.payload);
})
.addCase(createVul.rejected, (state, action) => {
  state.error = action.error.message;
})
// update: replace the old record with the updated one
.addCase(updateVul.fulfilled, (state, action) => {
  const i = state.items.findIndex((r) => r.id === action.payload.id);
  if (i !== -1) state.items[i] = action.payload;
})
.addCase(updateVul.rejected, (state, action) => {
  state.error = action.error.message;
})
// delete: remove the record with the matching id
.addCase(deleteVul.fulfilled, (state, action) => {
  state.items = state.items.filter((r) => r.id !== action.payload);
})
.addCase(deleteVul.rejected, (state, action) => {
  state.error = action.error.message;
});
```

2. Async Thunks with Axios

- Implement async thunks that call your FastAPI endpoints: fetch (GET list), create (POST), update (PUT /{id}), delete (DELETE /{id}).
- Handle success and error cases properly.

```

export const fetchVuls = createAsyncThunk(
  "vuls/fetch",
  async () => {
    const res = await axios.get(`${API}/vuls`, { withCredentials: true });
    return res.data;
  }
);

export const createVul = createAsyncThunk(
  "vuls/create",
  async (newRecord) => {
    const res = await axios.post(`${API}/vuls`, newRecord, { withCredentials: true });
    return res.data;
  }
);

export const updateVul = createAsyncThunk(
  "vuls/update",
  async ({ id, data }) => {
    const res = await axios.put(`${API}/vuls/${id}`, data, { withCredentials: true });
    return res.data;
  }
);

export const deleteVul = createAsyncThunk(
  "vuls/delete",
  async (id) => {
    await axios.delete(`${API}/vuls/${id}`, { withCredentials: true });
    return id;
  }
);

```

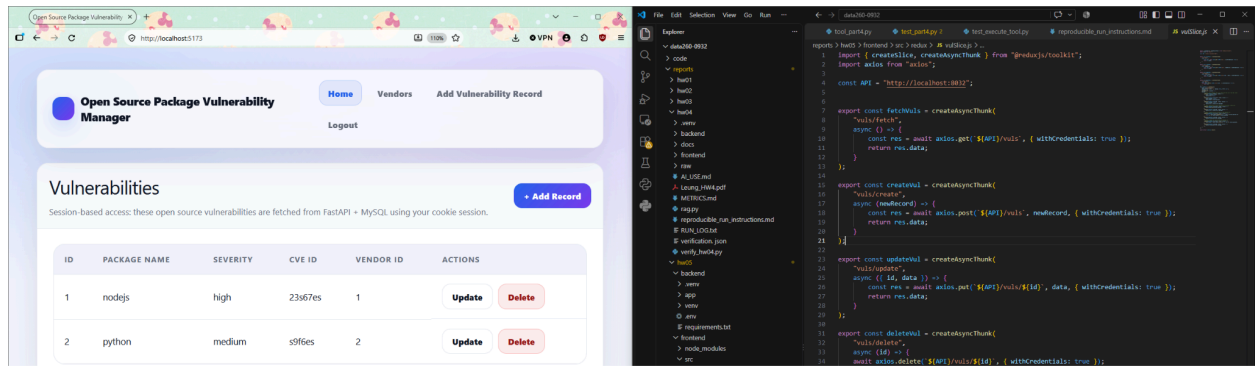
```

// fetch: show loading, then fill items with the list from the server
.addCase(fetchVuls.pending, (state) => {
  state.loading = true;
  state.error = null;
})
.addCase(fetchVuls.fulfilled, (state, action) => {
  state.loading = false;
  state.items = action.payload;
})
.addCase(fetchVuls.rejected, (state, action) => {
  state.loading = false;
  state.error = action.error.message;
})
// create: add the new record returned by the server to the list
.addCase(createVul.fulfilled, (state, action) => {
  state.items.push(action.payload);
})
.addCase(createVul.rejected, (state, action) => {
  state.error = action.error.message;
})
// update: replace the old record with the updated one
.addCase(updateVul.fulfilled, (state, action) => {
  const i = state.items.findIndex((r) => r.id === action.payload.id);
  if (i !== -1) state.items[i] = action.payload;
})
.addCase(updateVul.rejected, (state, action) => {
  state.error = action.error.message;
})
// delete: remove the record with the matching id
.addCase(deleteVul.fulfilled, (state, action) => {
  state.items = state.items.filter((r) => r.id !== action.payload);
})
.addCase(deleteVul.rejected, (state, action) => {
  state.error = action.error.message;
});

```

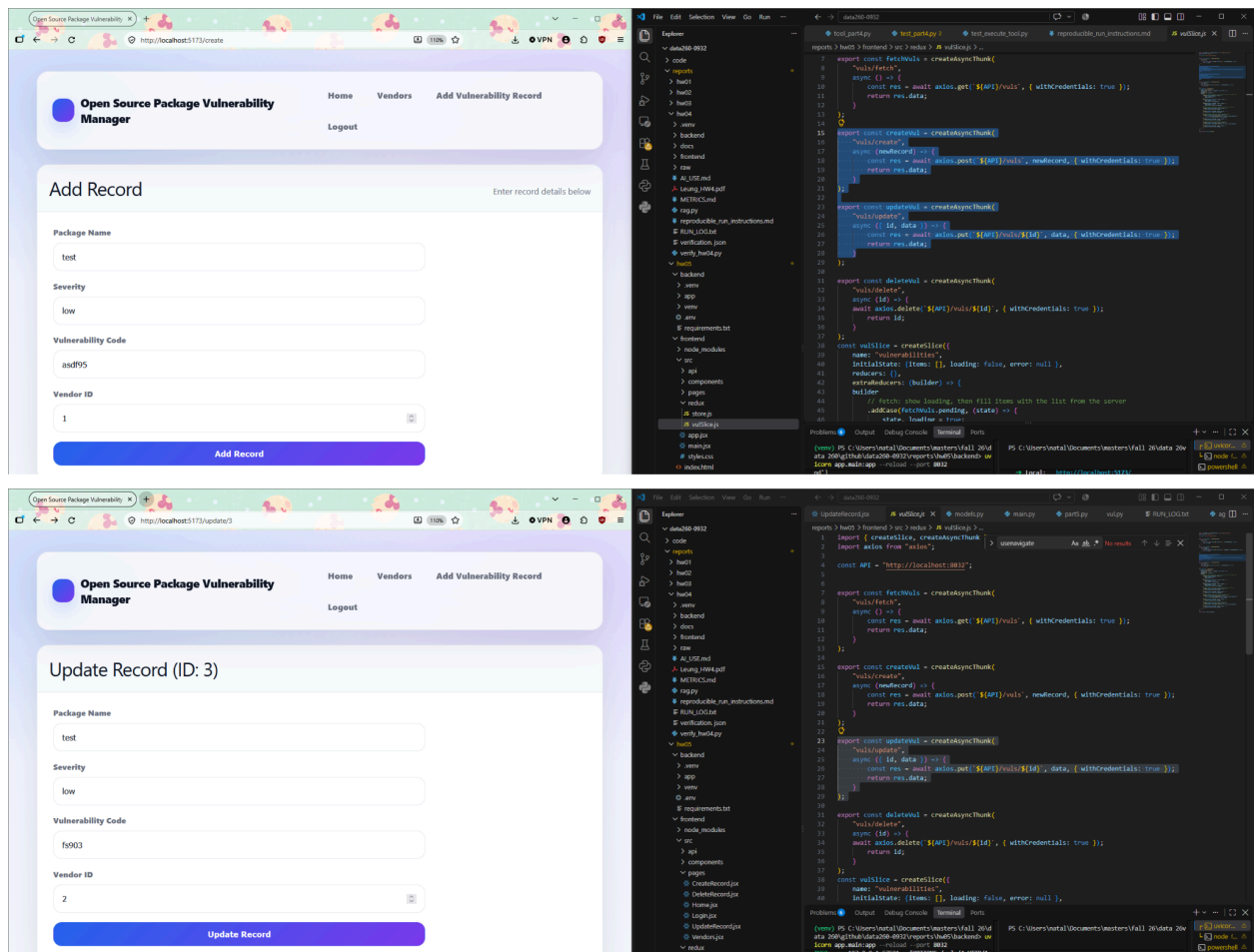
3. Home Screen

- Display all records by reading from Redux state; the list updates automatically when records are created, updated, or deleted.



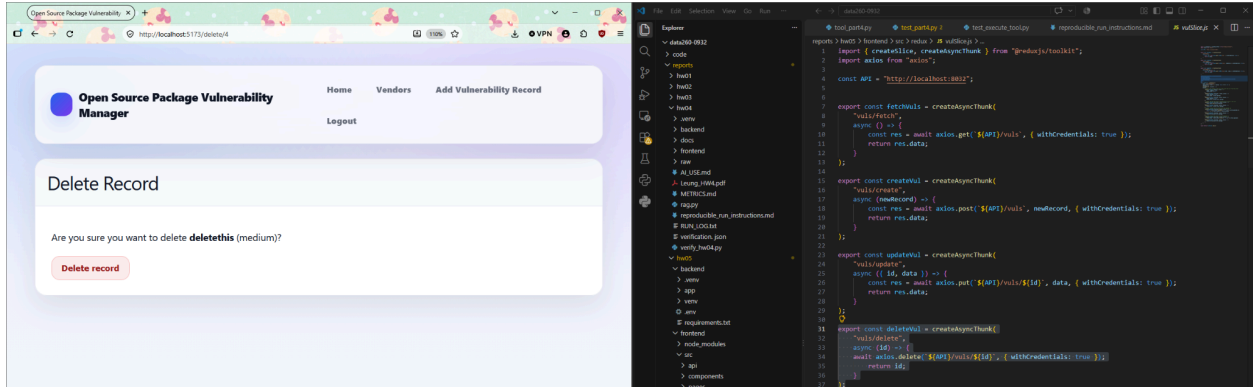
4. Create / Update Screens

- Build a form to create a new record and a form to update an existing record (select by ID).
- On submission, dispatch the correct Redux thunk and show the updated results on the Home Screen.



5. Delete Functionality

- Add a delete button next to each record in the list. On click, dispatch the delete thunk and update Redux state to remove the record.



Submit one screenshot per feature (Home, Create, Update, Delete) showing the code snippet (Redux thunk or slice) and the corresponding UI output together, in the same frame. Do not submit full code - only the relevant code snippet and working result.

Part 2 - MCP Tool Servers

6. search_meals_by_name(query, limit=5)

- Call TheMealDB Search meal by name endpoint: `.../search.php?s=<query>`
- Return up to limit meals with: name, area (cuisine), category, thumbnail, and the meal id for drill-down.

```
@mcp.tool()
def search_meals_by_name(query: str, limit: int = 5) -> list:
    try:
        response = requests.get("https://www.themealdb.com/api/json/v1/1/search.php?s=" + query, timeout=10)
        response.raise_for_status()
        data= response.json()
    except requests.exceptions.RequestException as e:
        logger.error("MealDB request failed: %s", e)
        raise RuntimeError("Cannot reach MealDB: " + str(e))
    except ValueError as e:
        logger.error("MealDB returned bad JSON: %s", e)
        raise RuntimeError("MealDB returned unreadable response")
    meals = data.get("meals")
    if not meals:
        logger.info("no matches for query: %s", query)
        return [] #for empty result
    results = []
    for meal in meals[:limit]:
        results.append({
            "id": meal.get("idMeal"),
            "name": meal.get("strMeal"),
            "area": meal.get("strArea"),
            "category": meal.get("strCategory"),
            "thumb": meal.get("strMealThumb")
        })
    return results
```

7. meals_by_ingredient(ingredient, limit=12)

- Call Filter by main ingredient: `.../filter.php?i=<ingredient>`
- Return small cards (name, thumbnail, id).

```

@mcp.tool()
def meals_by_ingredient(ingredient: str, limit: int = 12) -> list:
    try:
        response = requests.get("https://www.themealdb.com/api/json/v1/1/filter.php?i=" + ingredient, timeout=10)
        response.raise_for_status()
        data= response.json()
    except requests.exceptions.RequestException as e:
        logger.error("MealDB request failed: %s", e)
        raise RuntimeError("Cannot reach MealDB: " + str(e))
    except ValueError as e:
        logger.error("MealDB returned bad JSON: %s", e)
        raise RuntimeError("MealDB returned unreadable response")
    meals = data.get("meals")
    if not meals:
        logger.info("no matches for ingredient: %s", ingredient)
        return []
    results = []
    for meal in meals[:limit]:
        results.append({
            "id": meal.get("idMeal"),
            "name": meal.get("strMeal"),
            "thumb": meal.get("strMealThumb"),
        })
    return results

```

8. random_meal()

- Call Lookup a single random meal: .../random.php.

```

@mcp.tool()
def random_meal() -> dict:
    try:
        response = requests.get("https://www.themealdb.com/api/json/v1/1/random.php", timeout=10)
        response.raise_for_status()
        data= response.json()
    except requests.exceptions.RequestException as e:
        logger.error("MealDB request failed: %s", e)
        raise RuntimeError("Cannot reach MealDB: " + str(e))
    except ValueError as e:
        logger.error("MealDB returned bad JSON: %s", e)
        raise RuntimeError("MealDB returned unreadable response")
    meals = data.get("meals")
    if not meals:
        return {"message": "no matches"}
    return format_meal_details(meals[0])

```

9. meal_details(id)

Call Lookup a single meal by ID and return its full recipe details.

```
@mcp.tool()
def meal_details(id: str) -> dict:
    try:
        response = requests.get("https://www.themealdb.com/api/json/v1/1/lookup.php?i=" + str(id), timeout=10)
        response.raise_for_status()
        data= response.json()
    except requests.exceptions.RequestException as e:
        logger.error("MealDB request failed: %s", e)
        raise RuntimeError("Cannot reach MealDB: " + str(e))
    except ValueError as e:
        logger.error("MealDB returned bad JSON: %s", e)
        raise RuntimeError("MealDB returned unreadable response")
    meals = data.get("meals")
    if not meals:
        logger.info("no match for id: %s", id)
        return {"message": "no matches"}
    return format_meal_details (meals [0])
```

The API base is <https://www.themealdb.com/api/json/v1/1/>... and the published "test key" is 1, which is sufficient for this homework.

Set up your environment

10. Install the MCP Python SDK with CLI: `pip install "mcp[cli]"`

```
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw05\mcp> pip install "mcp[cli]==1.9.4"
26.7.22
Requirement already satisfied: httpcore==1.* in C:\Users\natal\AppData\Roaming\Python\Python314\site-packages (from httpx==0.27->mcp==1.9.4->mcp[cli]==1.9.4) (1.0.0)
Requirement already satisfied: h11>=0.16 in C:\Users\natal\AppData\Roaming\Python\Python314\site-packages (from httpcore==1.*->httpx==0.27->mcp==1.9.4->mcp[cli]==1.9.4) (0.16.0)
Requirement already satisfied: shellingham>=1.3.0 in C:\Users\natal\AppData\Roaming\Python\Python314\site-packages (from typer>=0.12.4->mcp[cli]==1.9.4) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in C:\Users\natal\AppData\Roaming\Python\Python314\site-packages (from typer>=0.12.4->mcp[cli]==1.9.4) (15.0.0)
Requirement already satisfied: annotated-doc>=0.0.2 in C:\Users\natal\AppData\Roaming\Python\Python314\site-packages (from typer>=0.12.4->mcp[cli]==1.9.4) (0.0.5)
Requirement already satisfied: colorama in C:\Users\natal\AppData\Roaming\Python\Python314\site-packages (from typer>=0.12.4->mcp[cli]==1.9.4) (0.4.6)
Requirement already satisfied: markdown-it-py>=2.2.0 in C:\Users\natal\AppData\Roaming\Python\Python314\site-packages (from rich>=13.8.0->typer>=0.12.4->mcp[cli]==1.9.4) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in C:\Users\natal\AppData\Roaming\Python\Python314\site-packages (from rich>=13.8.0->typer>=0.12.4->mcp[cli]==1.9.4) (2.21.0)
Requirement already satisfied: mdurl<=0.1 in C:\Users\natal\AppData\Roaming\Python\Python314\site-packages (from markdown-it-py>=2.2.0->rich>=13.8.0->typer>=0.12.4->mcp[cli]==1.9.4) (0.1.2)
Requirement already satisfied: click>=7.0 in C:\Users\natal\AppData\Roaming\Python\Python314\site-packages (from uvicorn>=0.23.1->mcp==1.9.4->mcp[cli]==1.9.4) (8.5.0)
Downloading mcp-1.9.4-py3-none-any.whl (130 kB)
Installing collected packages: mcp
  Attempting uninstall: mcp
    Found existing installation: mcp 1.30.0
    Uninstalling mcp-1.30.0:
      Successfully uninstalled mcp-1.30.0
  WARNING: The script mcp.exe is installed in 'C:\Users\natal\AppData\Roaming\Python\Python314\Scripts' which is not on PATH.
    Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed mcp-1.9.4
```

11. Write a single Python file (e.g., `meals_server.py`) that defines your tools and runs FastMCP over STDIO.

```
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

12. `search_meals_by_name(query, limit)`

- Input: query: str, limit: int (1–25)
- Output (list): objects with { id, name, area, category, thumb }

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=bbf1792c0ed4bb2e1bd2f90bf55e2f9df8b49e10a1b519917246e21f12e748f

meals Servers Tools Prompts Resources

Tools

Search tools...

☐ Paginated

search_meals_by_name

meals_by_ingredient

random_meal

meal_details

Results

```
1 {
2   "id": "52771",
3   "name": "Spicy Arrabiata Penne",
4   "area": "Italian",
5   "category": "vegetarian",
6   "thumb": "https://www.themeadb.com/images/media/mea",
7 }
```

Messages LEGACY

Newest First Clear Export

23:48:59 CLIENT → SERVER 556ms OK

TOOLS/CALL search_meals_by ↺ ⌵ ⌵ ⌵

Parameters:

```
1 {
2   "name": "search_meals_by_name",
3   "arguments": {
4     "limit": 5,
5     "query": "Arrabiata"
6   },
7   "meta": {
8     "progressToken": 7
9   }
10 }
```

Response:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 7,
4   "result": {
5     "content": [
6       {
7         "type": "text",
8         "text": "\n  \"id\": \"52771\", \n  \n"
9       }
10     ],
11     "isError": false
12   }
13 }
```

13. meals_by_ingredient(ingredient, limit)

- Input: ingredient: str, limit: int
- Output (list): objects with { id, name, thumb }

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=bbf1792c00ed4bb2e1bd2f90bf55e2f9df8b49e10a1b519917246e21f12e748f

meals Servers Tools Prompts Resources

Tools

Search tools...

Paginated

search_meals_by_name

meals_by_ingredient

random_meal

meal_details

Results

```
1 {
2   "id": "52940",
3   "name": "Brown Stew Chicken",
4   "thumb": "https://www.themealdb.com/images/media/meal_thumb/52940.jpg"
5 }
```

```
1 {
2   "id": "53161",
3   "name": "Chicken & chorizo rice pot",
4   "thumb": "https://www.themealdb.com/images/media/meal_thumb/53161.jpg"
5 }
```

```
1 {
2   "id": "52846",
3   "name": "Chicken & mushroom Hotpot",
4   "thumb": "https://www.themealdb.com/images/media/meal_thumb/52846.jpg"
5 }
```

```
1 {
2   "id": "52796",
3   "name": "Chicken Alfredo Primavera",
4   "thumb": "https://www.themealdb.com/images/media/meal_thumb/52796.jpg"
5 }
```

```
1 {
2   "id": "52934",
3   "name": "Chicken Basquaise",
4   "thumb": "https://www.themealdb.com/images/media/meal_thumb/52934.jpg"
5 }
```

```
1 {
2   "id": "52956",
3   "name": "Chicken Congee",
4   "thumb": "https://www.themealdb.com/images/media/meal_thumb/52956.jpg"
5 }
```

```
1 {
2   "id": "52795",
3   "name": "Chicken Handi",
4   "thumb": "https://www.themealdb.com/images/media/meal_thumb/52795.jpg"
5 }
```

```
1 {
2   "id": "52831",
3   "name": "Chicken Karaage",
4   "thumb": "https://www.themealdb.com/images/media/meal_thumb/52831.jpg"
5 }
```

```
1 {
2   "id": "53358",
3   "name": "Chicken Mandi",
4   "thumb": "https://www.themealdb.com/images/media/meal_thumb/53358.jpg"
5 }
```

```
1 {
2   "id": "53186",
3   "name": "Chicken with saffron, raisins & pine nuts",
4   "thumb": "https://www.themealdb.com/images/media/meal_thumb/53186.jpg"
5 }
```

```
1 {
2   "id": "53220",
3   "name": "Kabsa",
4   "thumb": "https://www.themealdb.com/images/media/meal_thumb/53220.jpg"
5 }
```

```
1 {
2   "id": "52813",
3   "name": "Kentucky Fried Chicken",
4   "thumb": "https://www.themealdb.com/images/media/meal_thumb/52813.jpg"
5 }
```

Messages LEGACY

Newest First Clear Export

23:50:25 CLIENT → SERVER 527ms OK

TOOLS/CALL meals_by_ingredient

Parameters:

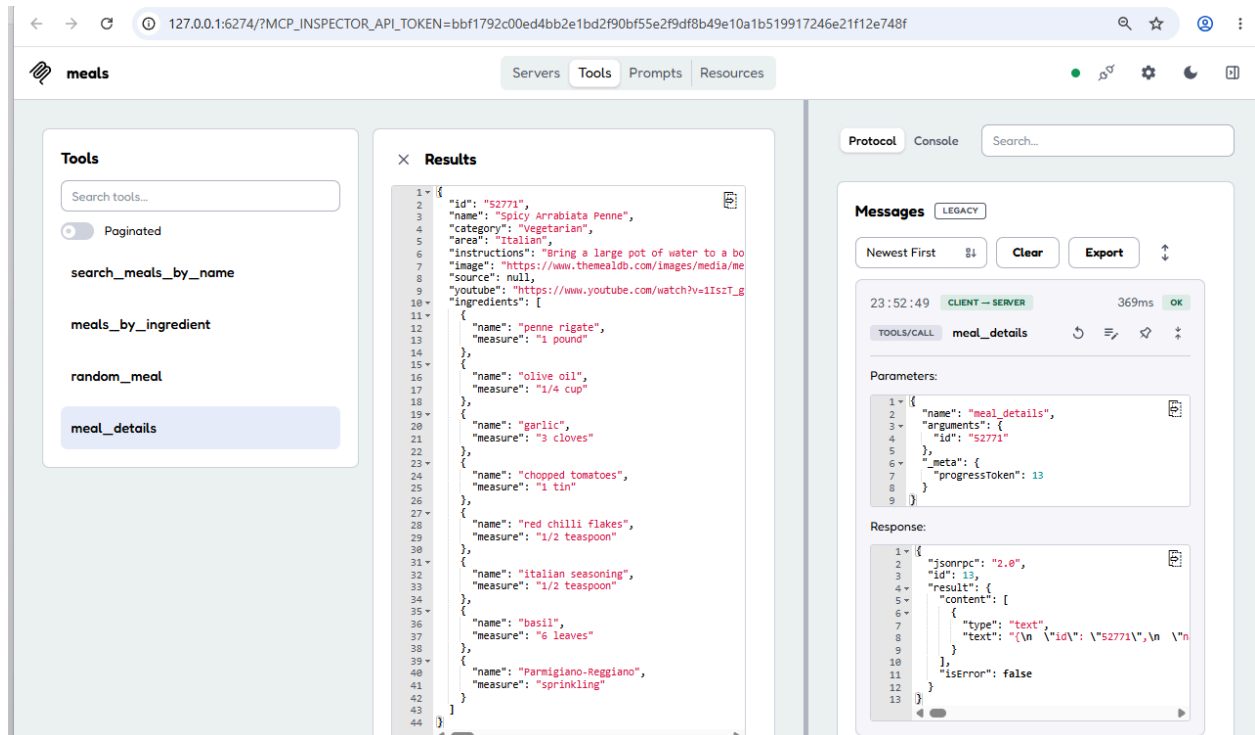
```
1 {
2   "name": "meals_by_ingredient",
3   "arguments": {
4     "limit": 12,
5     "ingredient": "chicken"
6   },
7   "meta": {
8     "progressToken": 8
9   }
10 }
```

Response:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 1,
4   "result": {
5     "content": [
6       {
7         "type": "text",
8         "text": "(\\n \\id\\": \\\"52940\\\",\\n \\n"
9       },
10      {
11        "type": "text",
12        "text": "(\\n \\id\\": \\\"53161\\\",\\n \\n"
13      },
14      {
15        "type": "text",
16        "text": "(\\n \\id\\": \\\"52846\\\",\\n \\n"
17      },
18      {
19        "type": "text",
20        "text": "(\\n \\id\\": \\\"52796\\\",\\n \\n"
21      },
22      {
23        "type": "text",
24        "text": "(\\n \\id\\": \\\"52934\\\",\\n \\n"
25      },
26      {
27        "type": "text",
28        "text": "(\\n \\id\\": \\\"52956\\\",\\n \\n"
29      },
30      {
31        "type": "text",
32        "text": "(\\n \\id\\": \\\"52795\\\",\\n \\n"
33      },
34      {
35        "type": "text",
36        "text": "(\\n \\id\\": \\\"52831\\\",\\n \\n"
37      },
38      {
39        "type": "text",
40        "text": "(\\n \\id\\": \\\"53358\\\",\\n \\n"
41      },
42      {
43        "type": "text",
44        "text": "(\\n \\id\\": \\\"53186\\\",\\n \\n"
45      },
46      {
47        "type": "text",
48        "text": "(\\n \\id\\": \\\"53220\\\",\\n \\n"
49      },
50      {
51        "type": "text",
52        "text": "(\\n \\id\\": \\\"52813\\\",\\n \\n"
53      }
54     ]
55   },
56   "isError": false
57 }
```

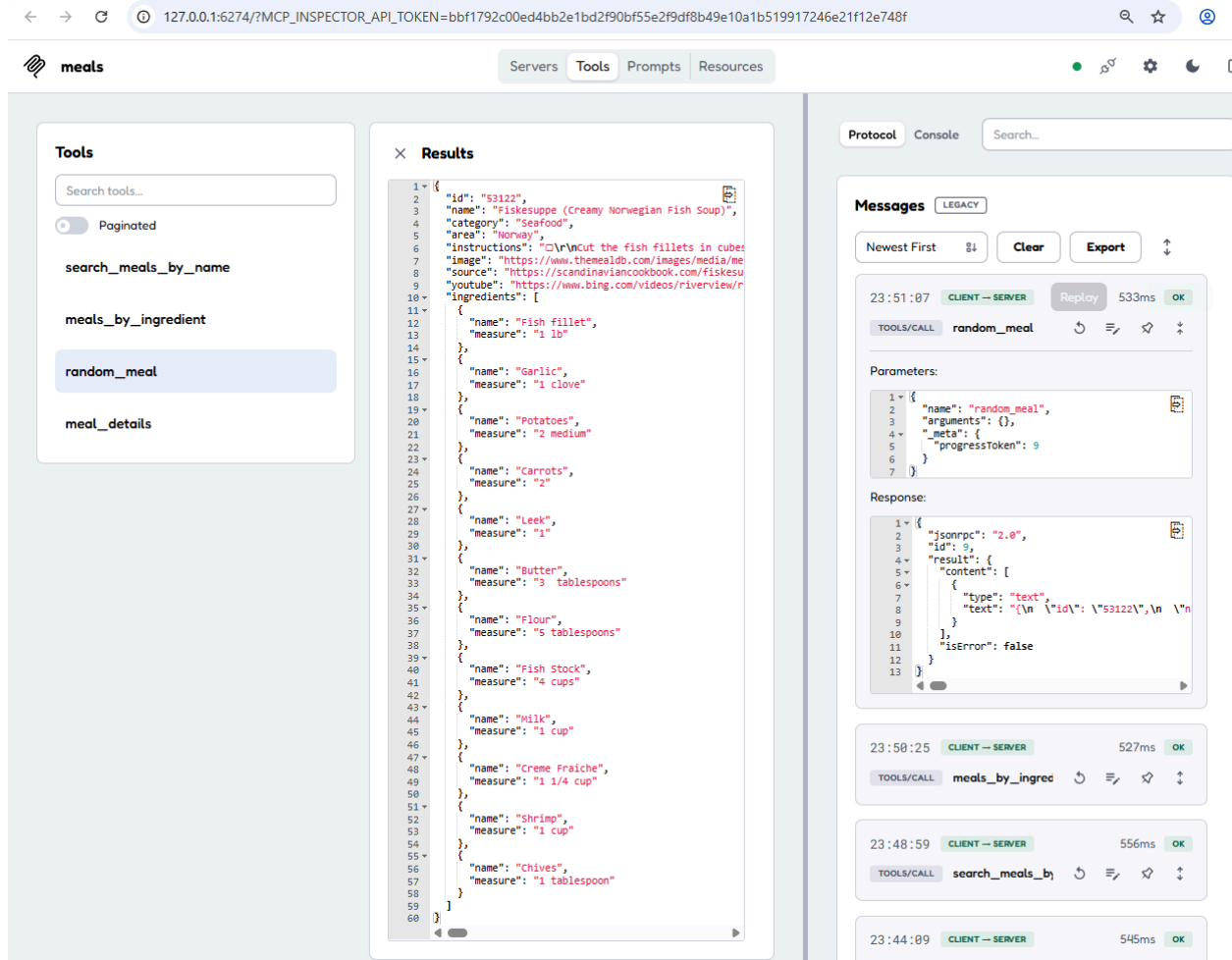
14. meal_details(id)

- Input: id: str | int
- Output (object): { id, name, category, area, instructions, image, source, youtube, ingredients: [{name, measure}] }



15. random_meal()

- Input: none
- Output: same shape as meal_details.
- Error handling: if an endpoint returns "meals": null (no results), return a clear, empty result with a short message (e.g., "no matches"). Network or JSON errors should raise a clean error to the client (MCP Inspector will display it). Run and debug locally with MCP Inspector



16. In your project folder, open a terminal and start development mode with the server file you wrote: `mcp dev <your_file_name.py>`

```
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw05\mcp> mcp dev .\meals.py
>>
C:\Users\natal\AppData\Roaming\Python\Python314\site-packages\pydantic_settings\sources\utils.py:47: IncompleteFieldDefinitionWarning: Field 'lifespan' has
an incomplete definition: its annotation contains an unresolved forward reference, so settings sources may fail to correctly resolve its value. Call `mode
l_rebuild()` on the model where the field is defined, once all the referenced types are defined.
  warnings.warn(
Starting MCP inspector...

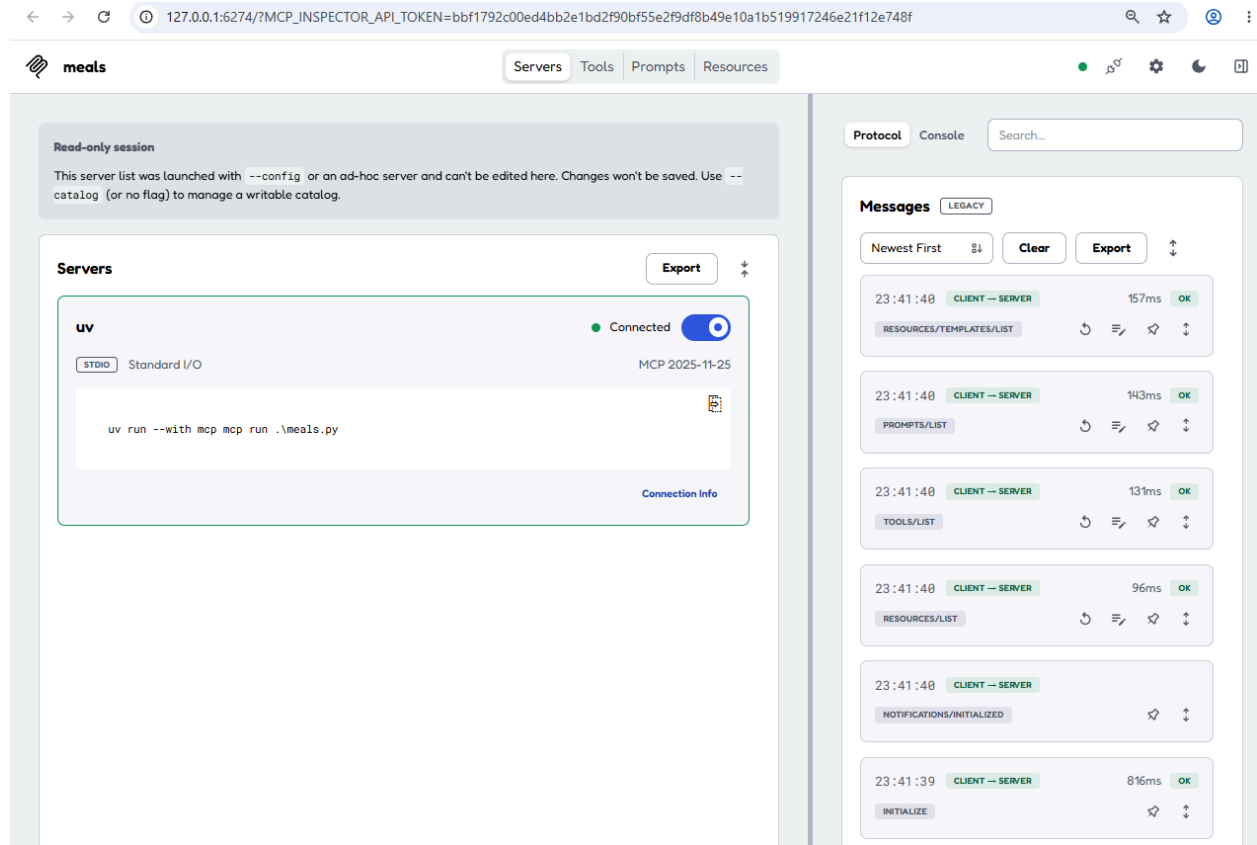
MCP Inspector Web is up and running at:
http://127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=bbf1792c00ed4bb2e1bd2f90bf55e2f9df8b49e10a1b519917246e21f12e748f

Sandbox (MCP Apps): http://127.0.0.1:6275/sandbox

Auth token: bbf1792c00ed4bb2e1bd2f90bf55e2f9df8b49e10a1b519917246e21f12e748f

Secrets: OS keychain

Opening browser...
|
```



17. The Inspector opens in your browser. Use it to call each tool with test inputs (e.g., "Arrabiata", ingredient "chicken"), verify outputs look like your designed schema, and fix any errors before moving on.

Output screenshots for this are above for the previous questions

Include one labeled MCP Inspector screenshot for each of the four tools, showing the input and returned output.

B. Domain MCP Server (required)

```
(venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw05\mcp> mcp dev .\vul.py
DATABASE: mysql+pymysql://root:seekrit@localhost:3306/s0932_rel
C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw05\mcp\venv\Lib\site-packages\pydantic_settings\sources\utils.py:47: IncompleteFieldDefinitionWarning: Field 'lifespan' has an incomplete definition: its annotation contains an unresolved forward reference, so settings sources may fail to correctly resolve its value. Call 'model_rebuild()' on the model where the field is defined, once all the referenced types are defined.
  warnings.warn(
Starting MCP Inspector...

MCP Inspector Web is up and running at:
http://127.0.0.1:6274/MCP_INSPECTOR_API_TOKEN=a456b5fdb97f087889ca80457f10fc42b43b185adb48bc4c4464a496ef7f01a

Sandbox (MCP Apps): http://127.0.0.1:6275/sandbox

Auth token: a456b5fdb97f087889ca80457f10fc42b43b185adb48bc4c4464a496ef7f01a

Secrets: OS keychain

Opening browser...
Terminate batch job (Y/N)? Y
```

- Build a second MCP server over your own <PREFIX>_rel domain data with exactly three tools: search, detail lookup, and one aggregate operation.

```
sys.path.append(os.path.join(os.path.dirname(__file__), "..", "backend"))
from app.database import SessionLocal
from app import models
```

```
reports > hw05 > backend > .env
```

```
1 DATABASE_URL=mysql+pymysql://root:seekrit@localhost:3306/s0932_rel
```

```
@mcp.tool()
def search_vulnerabilities(package_name: str) -> dict:
    if not package_name or not package_name.strip():
        return envelope(False, error="package_name must not be empty")

    db = get_session()
    try:
        rows = db.query(models.Vul).filter(models.Vul.package_name.ilike("%" + package_name + "%")).all()
        results = [{"id": r.id, "package_name": r.package_name, "severity": r.severity, "vul_code": r.vul_code, "vendor_id": r.vendor_id,} for r in rows]
        return envelope(True, data=results)
    except Exception as e:
        logger.error("search_vulnerabilities failed: %s", e)
        return envelope(False, error="database error")
    finally:
        db.close()

@mcp.tool()
def get_vulnerability(vuln_id: int) -> dict:
    db = get_session()
    try:
        row = db.query(models.Vul).filter(models.Vul.id == vuln_id).first()
        if not row:
            return envelope(False, error="vulnerability not found")
        data = {"id": row.id, "package_name": row.package_name, "severity": row.severity, "vul_code": row.vul_code, "vendor_id": row.vendor_id, "report_count": row.report_count}
        return envelope(True, data=data)
    except Exception as e:
        logger.error("get_vulnerability failed: %s", e)
        return envelope(False, error="database error")
    finally:
        db.close()

@mcp.tool()
def count_by_severity(min_count: int=0) -> dict:
    db = get_session()
    try:
        rows = db.query(models.Vul.severity, models.Vul.id).all()
        counts = {}
        if min_count < 0:
            return envelope(False, error="min_count has to be >= 0")
        for severity, _id in rows:
            counts[severity] = counts.get(severity, 0) + 1
        return envelope(True, data=counts)
    except Exception as e:
        logger.error("count_by_severity failed: %s", e)
        return envelope(False, error="database error")
    finally:
        db.close()
```

- Use one response envelope for all three tools: {ok, data, error}, where error is null on success. This is the same envelope shape you will reuse for execute_tool in Part 4 - define it once here and carry it through.

```
13 mcp = FastMCP("domain")
14 def envelope(ok, data=None, error=None):
15     return {"ok": ok, "data": data, "error": error}
```

- All MCP STDIO logs go to stderr, never stdout.

```
10 logging.basicConfig(stream=sys.stderr, level=logging.INFO)
11 logger = logging.getLogger("domain_server")
```

- Validate all three domain tools in MCP Inspector with at least one successful call and one intentionally invalid call per tool. Include screenshots of both - these rejected-call examples are the ones you'll reference again in Part 3.

Search vulnerabilities tool success

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=69712f58f8dddc42d2e8ded723762cd27b3256b5be8c9fb59a6f7b1380616dce

domain Servers Tools Prompts Resources

Tools

Search tools...

Paginated

search_vulnerabilities

get_vulnerability

count_by_severity

Results

```
1 {
2   "ok": true,
3   "data": [
4     {
5       "id": 1,
6       "package_name": "nodejs",
7       "severity": "high",
8       "vul_code": "23567es",
9       "vendor_id": 1
10    }
11  ],
12  "error": null
13 }
```

Protocol Console Search...

Messages LEGACY

Newest First Clear Export

02:58:43 CLIENT - SERVER 53ms OK

TOOLS/CALL search_vulnerabi

Parameters:

```
1 {
2   "name": "search_vulnerabilities",
3   "arguments": {
4     "package_name": "nodejs"
5   },
6   "meta": {
7     "progressToken": 5
8   }
9 }
```

Response:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 5,
4   "result": {
5     "content": [
6       {
7         "type": "text",
8         "text": "{\n  \"ok\": true,\n  \"data\"
9       }
10    ],
11    "isError": false
12  }
13 }
```

Search vulnerabilities tool error

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=d65c633e5df45f3479e9d9a9bd07ac94396a039cf6cf2bb638deedab9081625

domain Servers Tools Prompts Resources

Tools

Search tools...

Paginated

search_vulnerabilities

get_vulnerability

count_by_severity

Results

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "package_name must not be empty"
5 }
```

Protocol Console Search...

Messages LEGACY

Newest First Clear Export

20:58:12 CLIENT - SERVER 35ms OK

TOOLS/CALL search_vulnerabilities

Parameters:

```
1 {
2   "name": "search_vulnerabilities",
3   "arguments": {
4     "package_name": ""
5   },
6   "meta": {
7     "progressToken": 5
8   }
9 }
```

Response:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 6,
4   "result": {
5     "content": [
6       {
7         "type": "text",
8         "text": "{\n  \"ok\": false,\n  \"data\": null,\n  \"error\": \"packag
9       }
10    ],
11    "isError": false
12  }
13 }
```

Get vulnerability tool success

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=69712f58f8dddc42d2e8ded723762cd27b3256b5be8c9fb59a6f7b1380616dce

domain Servers Tools Prompts Resources

Tools

Search tools...

☐ Paginated

search_vulnerabilities

get_vulnerability

count_by_severity

Results

```
1 {
2   "ok": true,
3   "data": {
4     "id": 2,
5     "package_name": "python",
6     "severity": "medium",
7     "vul_code": "s9f6es",
8     "vendor_id": 2,
9     "report_count": 0
10  },
11   "error": null
12 }
```

Messages LEGACY

Newest First Clear Export

03:03:46 CLIENT → SERVER 43ms OK

TOOLS/CALL get_vulnerability

Parameters:

```
1 {
2   "name": "get_vulnerability",
3   "arguments": {
4     "vuln_id": 2
5   },
6   "meta": {
7     "progressToken": 7
8   }
9 }
```

Response:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 7,
4   "result": {
5     "content": [
6       {
7         "type": "text",
8         "text": "{\n  \"ok\": true,\n  \"data\"
9       }
10    ],
11    "isError": false
12  }
13 }
```

Get vulnerability tool error

127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=69712f58f8dddc42d2e8ded723762cd27b3256b5be8c9fb59a6f7b1380616dce

domain Servers Tools Prompts Resources

Connected (847ms)

Tools

Search tools...

☐ Paginated

search_vulnerabilities

get_vulnerability

count_by_severity

Results

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "vulnerability not found"
5 }
```

Messages LEGACY

Newest First Clear Export

03:04:08 CLIENT → SERVER 46ms OK

TOOLS/CALL get_vulnerability

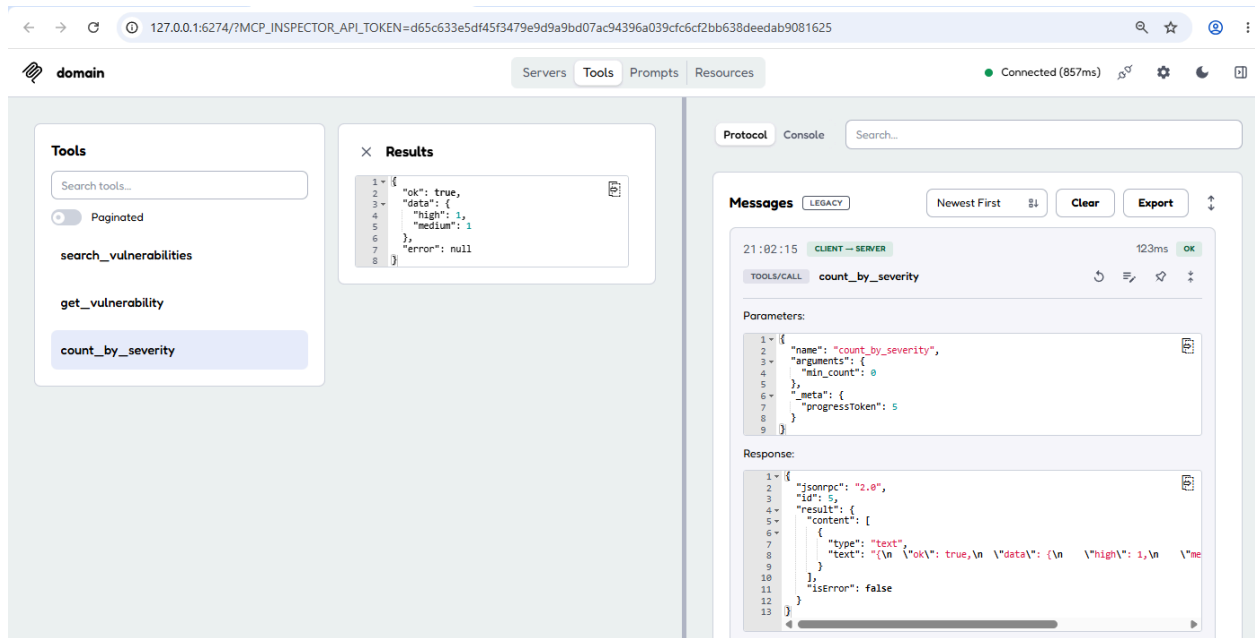
Parameters:

```
1 {
2   "name": "get_vulnerability",
3   "arguments": {
4     "vuln_id": 27
5   },
6   "meta": {
7     "progressToken": 8
8   }
9 }
```

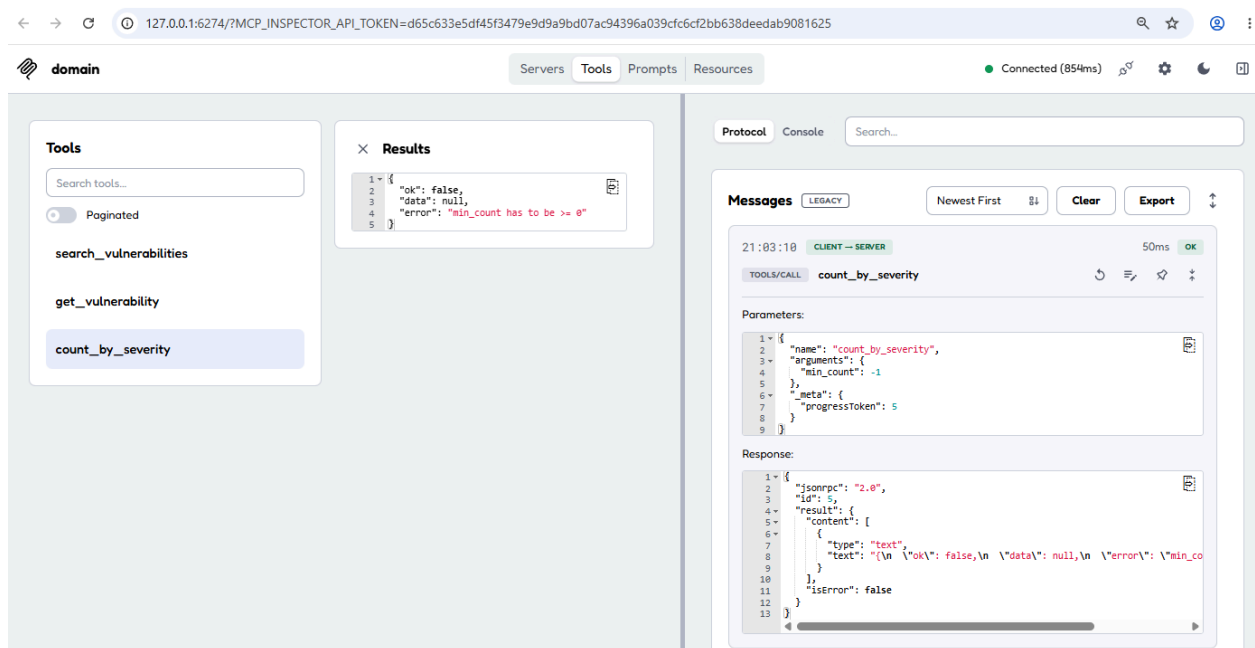
Response:

```
1 {
2   "jsonrpc": "2.0",
3   "id": 8,
4   "result": {
5     "content": [
6       {
7         "type": "text",
8         "text": "{\n  \"ok\": false,\n  \"data\": null,\n  \"error\": \"vulnerability not found\"
9       }
10    ],
11    "isError": false
12  }
13 }
```

Count by Severity tool success



Count by Severity tool error



Part 3 - Tool Contracts Under Stress

18. Using the same three domain tools and the same rejected-call examples you captured in Part 2B, Write down expected JSON input schema, the rejected JSON input, and the returned error output for each call. Documents why each Part 2B call was rejected - you do not need to re-run new rejected calls here.

Search_vulnerabilities

Expected input:

```
{
  "type": "object",
```

```
"properties": {  
  "package_name": { "type": "string", "minLength": 1, "description": "Package to search for must  
not be blank."}  
},  
"required": ["package_name"]  
}
```

Rejected input:

```
{ "package_name": " " }
```

Returned error

```
{ "ok": false, "data": null, "error": "package_name must not be empty" }
```

It was rejected because it expected a string value but got an whitespace instead. Because the tool trims whitespace, there is nothing to search.

Get_vulnerability

Expected input

```
{  
  "type": "object",  
  "properties": {  
    "vuln_id": { "type": "integer", "description": "ID of the vulnerability record" }  
  },  
  "required": ["vuln_id"]  
}
```

Rejected input

```
{ "vuln_id": 27 }
```

Returned error

```
{ "ok": false, "data": null, "error": "vulnerability not found" }
```

It was rejected because 27 does not match to any record ID. This then shows a not found error because record id 27 does not exist in the database.

Count_by_severity

Expected input

```
{  
  "type": "object",  
  "properties": {  
    "min_count": { "type": "integer", "minimum": 0, "description": "Minimum count a severity  
bucket needs to be included" }  
  }  
}
```

Rejected input

```
{ "min_count": -1 }
```

Returned error

```
{ "ok": false, "data": null, "error": "min_count has to be >= 0" }
```

It was rejected because the minimum count of severities cannot be negative. The tool limits the minimum count to be a valid 0. Anything less than 0 will return the error message.

19. Add timeouts and a bounded exponential-backoff retry policy to API or storage operations that may fail. Demonstrate (Show on report):

- Success on the first attempt.
- Failure on the first attempt followed by success after a retry.
- Failure after all allowed retries, returning a clean error result instead of crashing.

```
def call_retry(operation, max_retries=3, base_delay=0.2, timeout=2):
    attempt = 0

    while True:
        attempt += 1
        start = time.time()
        try:
            result = operation()
            elapsed = time.time() - start
            if elapsed > timeout:
                raise TimeoutError("attempt took " + str(round(elapsed, 3)) + "s, timeout is " + str(timeout) + "s")
            return {"ok": True, "data": result, "error": None, "attempts": attempt}

        except Exception as e:
            if attempt > max_retries:
                return {"ok": False, "data": None, "error": str(e), "attempts": attempt}
            delay = base_delay * (2 ** (attempt - 1))
            time.sleep(delay)

def fail_sequence(seed, failure_rate, count):
    rng = random.Random(seed)
    return [rng.random() < failure_rate for _ in range(count)]

def make_operation(fail_flags):
    flags = list(fail_flags)

    class Operation:
        def __call__(self):
            should_fail = flags.pop(0)

            if should_fail:
                raise ConnectionError("storage failed")
            return "query result"

    return Operation()

print("Success on the first attempt")
print(call_retry(make_operation([False]), max_retries=3, base_delay=0.1))

print("\nFailure on the first attempt followed by success after a retry")
print(call_retry(make_operation([True, False]), max_retries=3, base_delay=0.1))

print("\nFailure after all allowed retries, returning a clean error result instead of crashing")
print(call_retry(make_operation([True, True, True, True]), max_retries=3, base_delay=0.1))
```

```

PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw05\mcp> python .\part3_testing.py
DATABASE: mysql+pymysql://root:seekrit@localhost:3306/s0932_rel
Success on the first attempt
{'ok': True, 'data': 'query result', 'error': None, 'attempts': 1}

Failure on the first attempt followed by success after a retry
{'ok': True, 'data': 'query result', 'error': None, 'attempts': 2}

Failure after all allowed retries, returning a clean error result instead of crashing
{'ok': False, 'data': None, 'error': 'storage failed', 'attempts': 4}

```

20. Use VERIFY_SEED to reproducibly simulate failures at rates of 0%, 20%, and 50%. The same seed must produce the same success/failure sequence each time. Run 50 calls at each failure rate, for 150 calls total. For each rate, report the success rate, average latency, and p99 latency. Save the raw data for all 150 calls in reports/hw05/raw/, so your results can be double-checked.

```

def fail_sequence(seed, failure_rate, count):
    rng = random.Random(seed)
    return [rng.random() < failure_rate for _ in range(count)]

```

```

for rate in RATES:

    flags = fail_sequence(VERIFY_SEED, rate, CALLS_PER_RATE * 4)
    latencies = []
    successes = 0

    for call_number in range(1, CALLS_PER_RATE + 1):
        start = time.perf_counter()
        result = call_retry(make_db(flags), max_retries=3, base_delay=0.1, timeout=2)
        latency_ms = (time.perf_counter() - start) * 1000
        latencies.append(latency_ms)
        if result["ok"]:
            successes += 1
        raw_rows.append([rate, call_number, result["ok"], result["attempts"], round(latency_ms, 2)])

```

```

with open(os.path.join(RAW_DIR, "fault_injection_raw.csv"), "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["injected failure rate", "call number", "success", "attempts", "latency ms"])
    w.writerows(raw_rows)

```

```

PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw05\mcp> python .\part3_testing.py
DATABASE: mysql+pymysql://root:seekrit@localhost:3306/s0932_rel
Success on the first attempt
{'ok': True, 'data': 'query result', 'error': None, 'attempts': 1}

Failure on the first attempt followed by success after a retry
{'ok': True, 'data': 'query result', 'error': None, 'attempts': 2}

Failure after all allowed retries, returning a clean error result instead of crashing
{'ok': False, 'data': None, 'error': 'storage failed', 'attempts': 4}
2026-10-03 20:43:14 fault-injection measurement, VERIFY_SEED = 260932
2026-10-03 20:43:14 rate 0.0 success rate 1.0 mean ms 1.21 p99_ms 26.14
2026-10-03 20:43:15 rate 0.2 success rate 1.0 mean ms 21.01 p99_ms 102.06
2026-10-03 20:43:21 rate 0.5 success rate 0.92 mean ms 129.15 p99_ms 701.94

```

```
reports > hw05 > raw > fault_injection_raw.csv > data
```

	1	injected failure rate	call number	success	attempts	latency ms
125	0.5	24	True	2	101.51	
126	0.5	25	True	3	301.72	
127	0.5	26	True	2	101.35	
128	0.5	27	True	2	101.63	
129	0.5	28	True	1	0.8	
130	0.5	29	True	3	302.34	
131	0.5	30	True	1	0.98	
132	0.5	31	True	3	301.76	
133	0.5	32	True	1	0.79	
134	0.5	33	True	1	0.68	
135	0.5	34	True	1	0.67	
136	0.5	35	True	2	101.89	
137	0.5	36	True	1	0.73	
138	0.5	37	True	2	101.16	
139	0.5	38	True	1	0.75	
140	0.5	39	True	1	0.66	
141	0.5	40	True	2	101.7	
142	0.5	41	True	2	101.52	
143	0.5	42	True	1	0.66	
144	0.5	43	False	4	700.63	
145	0.5	44	True	1	1.4	
146	0.5	45	True	1	0.87	
147	0.5	46	True	1	0.78	
148	0.5	47	True	2	101.67	
149	0.5	48	True	2	101.52	
150	0.5	49	True	1	0.67	
151	0.5	50	True	1	0.62	
152						

21. Use your results table below to evaluate whether the retry policy is suitable for an interactive assistant. Then explain how you would modify the number of retries, timeout, or backoff delays for batch processing, where longer processing times are acceptable.

```
reports > hw05 > METRICS.md
```

	1	injected failure rate	success rate	mean latency ms	p99 latency ms
2	---	---	---	---	---
3		0.0	1.0	1.2	26.8
4		0.2	1.0	20.96	102.19
5		0.5	0.92	129.13	702.31

For an interactive assistant, the retry policy is suitable. It appears the 20% failure rate still shows the request succeeding, which is good. At 50% injected failure rate, there is a success rate decline and increase of latency. This is because of the retry try policy negatively impacting the response time. I think the best way is to modify it so there is a timeout for when there are failures that hit the safety rule block.

Additionally, for safety rule blocks, there should be fewer retries because it should straightforwardly tell the user the issue instead of retrying. For batch processing, I would have more retries because if users are not waiting for each request.

Part 4 - One Safe Tool Entry Point and Offline Tests

22. Create `execute_tool(name, inputs)`. This is the single function that the Part 5 agent will use to call any of the three domain tools. It must reuse the `{ok, data, error}` format from Part 2B, return the result as a JSON string, and handle errors without crashing.

```
def execute_tool(name, inputs):
    if name not in ALLOWED_TOOLS:
        result = vul.envelope(False, error="unknown tool: " + str(name))
        return json.dumps(result)

    tool_function = getattr(vul, name)
    try:
        result = tool_function(**inputs)
    except Exception as e:
        result = vul.envelope(False, error=str(e))

    return json.dumps(result)
```

23. Write a simple Python test runner using assert statements. For each of the three domain tools, test at least one valid input and the invalid input cases described in Part 3. Include at least six tests overall. Print PASS or FAIL for each test and a final X/Y summary.

```
# test 1: search_vulnerabilities, valid input
r = json.loads(tool_part4.execute_tool("search_vulnerabilities", {"package_name": "npm"}))
check("search_vulnerabilities valid input returns ok", r["ok"] is True and len(r["data"]) == 1)

# test 2: search_vulnerabilities, invalid input (blank package_name)
r = json.loads(tool_part4.execute_tool("search_vulnerabilities", {"package_name": ""}))
check("search_vulnerabilities blank input is rejected", r["ok"] is False)

# test 3: get_vulnerability, valid input
r = json.loads(tool_part4.execute_tool("get_vulnerability", {"vuln_id": 1}))
check("get_vulnerability valid id returns ok", r["ok"] is True and r["data"]["package_name"] == "npm")

# test 4: get_vulnerability, invalid input (id that does not exist)
r = json.loads(tool_part4.execute_tool("get_vulnerability", {"vuln_id": 27}))
check("get_vulnerability missing id is rejected", r["ok"] is False)

# test 5: count_by_severity, valid input
r = json.loads(tool_part4.execute_tool("count_by_severity", {"min_count": 0}))
check("count_by_severity valid input returns ok", r["ok"] is True)

# test 6: count_by_severity, invalid input (negative min_count)
r = json.loads(tool_part4.execute_tool("count_by_severity", {"min_count": -1}))
check("count_by_severity negative min_count is rejected", r["ok"] is False)

print(str(passed) + "/" + str(total) + "tests passed")
```

24. Use temporary fixtures or dependency injection so the tests run offline, repeatedly, and without an LLM.

```
testdb = "test.db"
if os.path.exists(testdb):
    os.remove(testdb)

test_engine = create_engine("sqlite:/// " + testdb)
Base.metadata.create_all(bind=test_engine)
TestSession = sessionmaker(bind=test_engine)
vul.get_session = TestSession
tool_part4.get_session=TestSession
db = TestSession()
vendor = models.Vendor(name="microsoft", industry="software", contact_email="microsoft@microsoft.org")
db.add(vendor)
db.commit()
db.refresh(vendor)
db.add(models.Vul(package_name="npm", severity="low", vul_code="fe9s5fs", vendor_id=vendor.id))
db.commit()
db.close()
```

```
[venv] PS C:\Users\Natal\Documents\masters\fall_26\data_260\github\data260-0932\reports\vh05\mcp> python .\test_part4.py
DATABASE: mysql+pymysql://root:seekrit@localhost:3306/s0932_rel
C:\Users\Natal\Documents\masters\fall_26\data_260\github\data260-0932\reports\vh05\mcp\venv\lib\site-packages\pydantic_settings\sources\utils.py:47: IncompleteFieldDefinitionWarning: Field 'lifespan' has an incomplete definition: its annotation contains an unresolved forward reference, so settings sources may fail to correctly resolve its value. Call 'model_rebuild()' on the model where the field is defined, once all the referenced types are defined.
  warnings.warn(
PASS - search_vulnerabilities valid input returns ok
PASS - search_vulnerabilities blank input is rejected
PASS - get_vulnerability valid id returns ok
PASS - get_vulnerability missing id is rejected
PASS - count_by_severity valid input returns ok
PASS - count_by_severity negative min_count is rejected
6/6tests passed
```

Include one screenshot of the terminal running the Part 4 test runner. The screenshot must show the PASS/FAIL result for each test and the final X/Y summary.

Part 5 - Agent Loop, Safety Rule, and Reflection

This part adds a business-logic safety rule and connects the tested tool layer to an Ollama-based agent.

I. Safety rule

- Add one domain-specific safety rule inside `execute_tool`.
- When the rule is violated, return `{ok: false, data: null, error: "..."} without raising an exception.`
- Demonstrate one allowed call and one blocked call.

```
def violates_safety_rule(name, inputs):
    if name == "search_vulnerabilities":
        term = str(inputs.get("package_name", "")).strip()
        if 0 < len(term) < 3:
            return ("SAFETY: search term must be at least " + str(3) + "characters (got \"" + term + "\")")
        return None

def execute_tool(name, inputs):
    if name not in ALLOWED_TOOLS:
        result = vul.envelope(False, error="unknown tool: " + str(name))
        return json.dumps(result)
    block_reason = violates_safety_rule(name, inputs)
    if block_reason:
        return json.dumps(vul.envelope(False, error=block_reason))
    tool_function = getattr(vul, name)
    try:
        result = tool_function( ** inputs)
    except Exception as e:
        result = vul.envelope(False, error=str(e))

    return json.dumps (result)
```

```
• (venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw05\mcp> $env:PYTHON
PATH="..\backend"
• (venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw05\mcp> python .\pa
rt5.py
DATABASE: mysql+pymysql://root:seekrit@localhost:3306/s0932_rel
C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw05\mcp\venv\Lib\site-packages
\pydantic_settings\sources\utils.py:47: IncompleteFieldDefinitionWarning: Field 'lifespan' has an incomplete
definition: its annotation contains an unresolved forward reference, so settings sources may fail to correctl
y resolve its value. Call `model_rebuild()` on the model where the field is defined, once all the referenced
types are defined.
  warnings.warn(
scenario | steps | tool_calls | stop_reason
How many vulnerabilities are High severity? | 1 | 0 | normal_completion
Tell me about the vulnerability with id 1. | 2 | 1 | normal_completion
Call search_vulnerabilities tool where package_name is 'np'. | 1 | 1 | safety_rule_block
What is bread for? | 1 | 0 | normal_completion
```

II. Agent loop

- Implement run_agent(user_input) using the local Ollama model and the three domain tools.
- Add a turn counter and max_steps limit.
- Log each step, tool call, input, result, and final stop reason to agent_runs.jsonl.

```

def run_agent(user_input, model_fn=None, max_steps=5, log_path="agent_runs.json1"):
    if model_fn is None:
        model_fn = ask_ollama
    conversation = SYSTEM_INSTRUCTIONS + "\nUser question: " + user_input + "\n"
    steps_log = []
    stop_reason = None
    final_answer = None
    step = 0
    while step < max_steps:
        step += 1
        reply = model_fn(conversation).strip()
        tool_call = try_parse_tool_call(reply)
        if tool_call is None:
            final_answer = reply
            stop_reason = "normal_completion"
            steps_log.append({"step": step, "tool": None, "inputs": None, "result": None, "model_reply": reply})
            break
        name, inputs = tool_call
        result = json.loads(tool_part4.execute_tool(name, inputs))
        steps_log.append({"step": step, "tool": name, "inputs": inputs, "result": result})
        if not result["ok"] and str(result["error"]).startswith("SAFETY:"):
            stop_reason = "safety_rule_block"
            final_answer = "I can't do that: " + str(result["error"])
            break
        conversation += "\nTool " + name + " returned " + json.dumps(result) + "\n"
    if stop_reason is None:
        stop_reason = "max_steps_reached"
        final_answer = "Stopped after reaching the " + str(max_steps) + "-step limit."
    record = {
        "time": time.strftime("%Y-%m-%d %H:%M: %S"),
        "user_input": user_input,
        "steps": steps_log,
        "stop_reason": stop_reason,
        "final_answer": final_answer,
    }
    with open(log_path, "a") as f:
        f.write(json.dumps(record) + "\n")

```

```

reports > hwd05 > mcp > E agent_runs.json1
1 {"time": "2026-10-05 02:33: 27", "user_input": "How many vulnerabilities are High severity?", "steps": [{"step": 1, "tool": null, "inputs": null, "result": null, "model_reply": "The tool"}
2 {"time": "2026-10-05 02:33: 35", "user_input": "Tell me about the vulnerability with id 1.", "steps": [{"step": 1, "tool": "get_vulnerability", "inputs": {"vuln": 1}, "result": {"id": 1, "severity": "High", "description": "A remote code execution vulnerability in the vulnerable application."}}, {"step": 2, "tool": null, "inputs": null, "result": null, "model_reply": "The tool returned the following information about vulnerability with id 1: {'id': 1, 'severity': 'High', 'description': 'A remote code execution vulnerability in the vulnerable application.'}"}]}
3 {"time": "2026-10-05 02:33: 40", "user_input": "Call search_vulnerabilities tool where package_name is 'np'.", "steps": [{"step": 1, "tool": "search_vulnerabilities", "inputs": {"package_name": "np"}, "result": {"package_name": "np", "vulnerabilities": [{"id": 1, "severity": "High", "description": "A remote code execution vulnerability in the vulnerable application."}, {"id": 2, "severity": "Medium", "description": "A buffer overflow vulnerability in the vulnerable application."}]}]}, {"step": 2, "tool": null, "inputs": null, "result": null, "model_reply": "The tool returned the following information about vulnerabilities for package_name 'np': [{'id': 1, 'severity': 'High', 'description': 'A remote code execution vulnerability in the vulnerable application.'}, {'id': 2, 'severity': 'Medium', 'description': 'A buffer overflow vulnerability in the vulnerable application.'}]}]}
4 {"time": "2026-10-05 02:33: 46", "user_input": "What is bread for?", "steps": [{"step": 1, "tool": null, "inputs": null, "result": null, "model_reply": "The tool returned the following information about bread: {'id': 1, 'severity': 'High', 'description': 'A remote code execution vulnerability in the vulnerable application.'}"}]}
5

```

III. Additional tests

After implementing the Part 5 safety rule and agent loop, add two tests to the existing Part 4 offline test suite. Test that `execute_tool` blocks a call that violates the safety rule, and that `run_agent`, using `MockModel`, stops after reaching `max_steps`. Both tests must run offline without an API key, live model, external API, or database.

```

def violates_safety_rule(name, inputs):
    if name == "search_vulnerabilities":
        term = str(inputs.get("package_name", "")).strip()
        if 0 < len(term) < 3:
            return ("SAFETY: search term must be at least " + str(3) + "characters (got \"" + term + "\")")
    return None

def execute_tool(name, inputs):
    if name not in ALLOWED_TOOLS:
        result = vul.envelope(False, error="unknown tool: " + str(name))
        return json.dumps(result)
    block_reason = violates_safety_rule(name, inputs)
    if block_reason:
        return json.dumps(vul.envelope(False, error=block_reason))
    tool_function = getattr(vul, name)
    try:
        result = tool_function( ** inputs)
    except Exception as e:
        result = vul.envelope(False, error=str(e))

    return json.dumps (result)

if __name__ == "__main__":
    print("Allowed call (3+ characters):")
    print(execute_tool("search_vulnerabilities", {"package_name": "lodash"}))
    print("\nBlocked call (too short, violates the safety rule):")
    print(execute_tool("search_vulnerabilities", {"package_name": "ab"}))

```

```

# test 7: safety rule
r = json.loads(tool_part4.execute_tool("search_vulnerabilities", {"package_name": "ab"}))
check("execute_tool blocks a call that violates the safety rule", r["ok"] is False and r["error"].startswith("SAFETY:"))
def mock_model_always_calls_tool(prompt):
    return json.dumps({"tool": "get_vulnerability", "inputs": {"vuln_id": 1}})

# test 8: run_agent
test_log = "test_agent_runs.jsonl"
result = run_agent("test question", model_fn=mock_model_always_calls_tool, max_steps=3, log_path=test_log)
check("run_agent stops after reaching max_steps",
result["stop_reason"] == "max_steps_reached" and len(result["steps"]) == 3)
if os.path.exists(test_log):
    os.remove(test_log)

```

```

for text in SCENARIOS:
    result = run_agent(text, max_steps=5)
    tool_calls = sum(1 for s in result["steps"] if s["tool"])
    print(text, "|", len(result["steps"]), "|", tool_calls, "|", result["stop_reason"])

```

```

testdb = "test.db"
if os.path.exists(testdb):
    os.remove(testdb)

test_engine = create_engine("sqlite:/// " + testdb)
Base.metadata.create_all(bind=test_engine)
TestSession = sessionmaker(bind=test_engine)
vul.get_session = TestSession
tool_part4.get_session=TestSession
db = TestSession()
vendor = models.Vendor(name="microsoft", industry="software", contact_email="microsoft@microsoft.org")
db.add(vendor)
db.commit()
db.refresh(vendor)
db.add(models.Vul(package_name="npm", severity="low", vul_code="fe9s5fs", vendor_id=vendor.id))
db.commit()
db.close()

```

```

PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw05\mcp> python .\test_part4.py
DATABASE: mysql+pymysql://root:seekrit@localhost:3306/s0932_rel
C:\Users\natal\AppData\Roaming\Python\Python314\site-packages\pydantic_settings\sources\utils.py:47: IncompleteFieldDefinitionWarning: Field 'lifespan' has an incomplete definition: its annotation contains an unresolved forward reference, so settings sources may fail to correctly resolve its value. Call `model_rebuild()` on the model where the field is defined, once all the referenced types are defined.
  warnings.warn(
PASS - search_vulnerabilities valid input returns ok
PASS - search_vulnerabilities blank input is rejected
PASS - get_vulnerability valid id returns ok
PASS - get_vulnerability missing id is rejected
PASS - count_by_severity valid input returns ok
PASS - count_by_severity negative min_count is rejected
PASS - execute_tool blocks a call that violates the safety rule
PASS - run_agent stops after reaching max_steps
8/8tests passed

```

IV. Metrics

- Run at least four scenarios using the local Ollama model.

```

SCENARIOS = [
    "How many vulnerabilities are High severity?",
    "Tell me about the vulnerability with id 1.",
    "Call search_vulnerabilities tool where package_name is 'np'.",
    "What is bread for?",
]

print("scenario | steps | tool_calls | stop_reason")
for text in SCENARIOS:
    result = run_agent(text, max_steps=5)
    tool_calls = sum(1 for s in result["steps"] if s["tool"])
    print(text, "|", len(result["steps"]), "|", tool_calls, "|", result["stop_reason"])

```

```

• (venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw05\mcp> $env:PYTHON
PATH="..\backend"
• (venv) PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw05\mcp> python .\pa
rt5.py
DATABASE: mysql+pymysql://root:seekrit@localhost:3306/s0932_rel
C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw05\mcp\venv\Lib\site-packages
\pydantic_settings\sources\utils.py:47: IncompleteFieldDefinitionWarning: Field 'lifespan' has an incomplete
definition: its annotation contains an unresolved forward reference, so settings sources may fail to correctl
y resolve its value. Call `model_rebuild()` on the model where the field is defined, once all the referenced
types are defined.
  warnings.warn(
scenario | steps | tool_calls | stop_reason
How many vulnerabilities are High severity? | 1 | 0 | normal_completion
Tell me about the vulnerability with id 1. | 2 | 1 | normal_completion
Call search_vulnerabilities tool where package_name is 'np'. | 1 | 1 | safety_rule_block
What is bread for? | 1 | 0 | normal_completion

```

- Record each scenario's step count, stop reason, and tool-call count in METRICS.md and in report too.

```

reports > hw05 > METRICS.md
6
7 | scenario | steps | tool_calls | stop_reason|
8 | --- | --- | --- | --- |
9 How many vulnerabilities are High severity? | 1 | 0 | normal_completion|
10 Tell me about the vulnerability with id 1. | 2 | 1 | normal_completion|
11 Call search_vulnerabilities tool where package_name is 'np'. | 1 | 1 | safety_rule_block|
12 What is bread for? | 1 | 0 | normal_completion|

```

V. Reflection

Write a 200–300-word reflection in REFLECTION.md and report too:

pick one run from your agent_runs.jsonl-style log and explain, step by step, what the harness did and why it stopped (normal completion, safety-rule block, or max_steps ceiling).

Looking at the third run, it says where the user asked to "Call search_vulnerabilities tool where package_name is "np". The harness passed to use the specified tool and look up np, which is following exactly as asked. Because I specified for the model to not change the question, it did not hallucinate.

Before running the search, the tool layer had to validate the input by looking if the "np" follows the safety rule. The safety rule requires search terms to be at least three characters. Because "np" is less than three characters, the tool returned ok: false, data:null, and error noting the safety. The run was stopped because the harness logged that one step and set the stop reason to be the safety rule. Therefore, it did not return an error to the model and there were no retries. I had told the model to say "I can't do that" if it cannot complete the request. This was followed. The stop reason was not normal completion because the model did make a response itself as it followed the model rule. It was not the max steps ceiling too because the step hit step one and not the limit.

AI_USE.md

reports > hw05 > AI_USE.md

- 1 1. I used the AI assistant to help debug the redux slicing and tool calling. There kept showing errors of app. database not being able to be reached and tools not being able to be called correctly. I used AI to help debug that and found the pathing to be an issue. If I set the environment variable path to the backend and then called the files that used the app.database, then it was able to work. I was also able to just call the whole tool file instead of specifying the execute_tool function because that helped resolve the pathing issue as well. I did the mcp for the mealmd and vul files as I have experience in my internships working with mcp files. I did have to use AI to help figure out the mcp inspector because the newest version has a different UI and work around the version 1.0 that I was more familiar with. I did the agent myself and the vulnerability and vendor implementation as well. I did use AI to debug the issue with vendor not being able to be updated.
- 2 2. One AI produced output that was wrong was it helping me debug the update vulnerability/vendors error. The error was that nothing changes to the homepage when I update a vulnerability. It told me to fix the vendor_id datatype to be a number instead of a string when I already had that. It told me the error on Postman was probably a 422 error but it did not match to the issue I had.
- 3 3. I fixed the previous problem by looking into the logs of the frontend and backend terminals. I also looked at the console output for the website itself. It showed there was a 404 error. The vendor did not exist, so I looked at the relationship I had implemented and fixed that. There was a problem with my implementation of the relationship so I use AI assistant to help me debug.
- 4 4. The relationship query I used a try and except for if it could not get the vendor I raised the error 404 automatically. I fixed it so that if the vendor does not exist then I raise the 404 error. I used the try except to try to create the vulnerability and used an integrityerror to raise 409 response if the vulnerability code existed. When testing again I found the issue was that the vulnerability code existed. This is because this field is to satisfy the requirement that the vulnerability code is unique. I was able to properly raise the correct http error and fix my vulnerability code to be unique in the form and it worked.